

Step 1 — A Normal Graph With Only Short-Term (Checkpoint) Memory — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real LangGraph calls, in the right order), **Advanced** (what actually proves the checkpoint's limitation, not just its success). Read Basic first even if you already built Project 3 — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — What a checkpoint actually is, and what it isn't](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What a checkpointer actually is, and what it isn't

Basic Version

A checkpointer saves a conversation's state under a `thread_id`. As long as you keep using the same `thread_id`, it's the same conversation, even across separate runs of your program. The moment you use a different `thread_id`, it's a brand-new conversation with brand-new, empty state — the checkpointer has no idea the other `thread_id` even exists.

Things to use: - `TypedDict` for your state shape. - `langgraph.graph.add_messages` as the reducer for your `messages` field. - `sqlite3.connect(...)` plus `SqliteSaver` from `langgraph.checkpoint.sqlite`. - `graph.invoke(input, config={"configurable": {"thread_id": "..."}})`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exact pieces:

- `from typing import Annotated; from langgraph.graph.message import add_messages` — your `messages` field needs this reducer, or a second `invoke` call on the same thread would overwrite the first message list instead of appending to it.
- `from langgraph.checkpoint.sqlite import SqliteSaver and import sqlite3` — `SqliteSaver` needs a real, open `sqlite3.Connection`, not just a file path string. Build it with `sqlite3.connect("memory_keeper.db", check_same_thread=False)` — `check_same_thread=False` matters because `LangGraph` may touch the connection from a different thread than the one that opened it.
- `builder.compile(checkpointer=checkpointer)` — this is the one line that turns a plain graph into one whose state survives between calls.
- `graph.invoke({"messages": [{"role": "user", "content": text}]}, config={"configurable": {"thread_id": thread_id}})` — the `config` dict, with `thread_id` inside `configurable`, is what tells the checkpointer which conversation this call belongs to.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think carefully about what actually proves a checkpointer is working, versus what only looks like it's working. If you build the graph once and call `.invoke()` on it twice in the same Python process, without ever restarting anything, of course it remembers — the graph object, the model, everything is still sitting in memory regardless of whether a checkpointer exists at all. That test proves nothing about persistence.

The real test is two **separate process invocations** of `main.py` — run it, chat, let the process exit completely, then run `python main.py` again as a fresh process, and confirm the same `thread_id` still recalls the earlier messages. Only a file-backed checkpointer (`SqliteSaver`, pointed at a real `.db` file) can pass that test. This is exactly why the Setup step has you install `langgraph-checkpoint-sqlite` instead of relying on the default in-memory checkpointer.

The second thing worth proving on purpose, not by accident: a different `thread_id` genuinely starts from nothing. Don't just assume this — actually run it, and actually read the output. If your test only ever uses one `thread_id`, you've proven the checkpointer works, but you haven't yet proven you understand *why* Step 2 is needed at all.

Things to try before Hint 2: - Run `main.py`, say "my name is Sam," exit, run it again with the same `thread_id`, ask "what's my name?" — confirm it answers correctly. - Run it a third time with a different `thread_id`, ask "what's my name?" — confirm it has no idea, and says so honestly instead of guessing.

Difference between Basic, Intermediate, and Advanced: Basic names the pieces and the one-sentence rule (same `thread_id` = same conversation). Intermediate gives the exact imports and call shapes. Advanced is about designing a test that actually proves persistence across real process restarts, and deliberately proving the "different thread, no memory" half too — not just the half that makes the checkpointer look good.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state: messages (a growing list)

chat_node(state):
    call the model on state["messages"]
    return the reply, appended to messages

graph: START -> chat_node -> END
compiled with a real, file-backed checkpointer

main.py:
    pick a thread_id
    loop: read a line, invoke the graph with that thread_id, print the reply
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Here's almost the whole thing — type it out yourself and adjust to your own file names:

```
# state.py
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages
```

```
class ChatState(TypedDict):
    messages: Annotated[list, add_messages]
```

```
# graph.py
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from state import ChatState

model = ChatOpenAI(model="gpt-4o-mini")

def chat_node(state: ChatState) -> dict:
    reply = model.invoke(state["messages"])
    return {"messages": [reply]}

def build_graph(checkpointer):
    builder = StateGraph(ChatState)
    builder.add_node("chat_node", chat_node)
    builder.add_edge(START, "chat_node")
    builder.add_edge("chat_node", END)
    return builder.compile(checkpointer=checkpointer)
```

Your turn: write `main.py` 's checkpointer setup and terminal loop, using `SqliteSaver` and a fixed `thread_id`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the two-process test yourself, using this skeleton:

```
# main.py
import sqlite3
import sys
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph

THREAD_ID = "sam-thread-1" # change this by hand between test runs

def main():
    conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)
    checkpointer = SqliteSaver(conn)
    graph = build_graph(checkpointer)

    print(f"MemoryKeeper -- thread_id={THREAD_ID}. Type 'exit' to quit.")
    while True:
        text = input("you: ")
        if text.strip().lower() == "exit":
            break
        # your turn: build the config dict with thread_id, invoke the graph,
        # print the last message's content

if __name__ == "__main__":
    main()
```

Notice `THREAD_ID` is hardcoded at the top for now, on purpose — Step 2 replaces this with a real per-run generated ID. Keeping it hardcoded and easy to change by hand is exactly what makes the "same thread vs. different thread" test in Hint 1's Advanced section easy to actually run.

Compare your finished graph, state, and loop against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic sketches the whole shape in plain words. Intermediate gives the real, nearly-complete `StateGraph` and node code. Advanced is about the test discipline this step actually exists to teach — proving persistence across real separate process runs, and proving the limitation (a new thread forgets everything) just as deliberately as you prove the feature (a repeated thread remembers).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)